

**60**

**Advanced  
SQL Queries**

01

How to find the second-highest salary of an employee?

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```



02

How to find employees who joined the company in the same month and year as their manager?

```
SELECT e.employee_id, e.name
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id
WHERE MONTH(e.join_date) = MONTH(m.join_date)
AND YEAR(e.join_date) = YEAR(m.join_date);
```



03

Write a query to count the number of employees whose names start and end with the same letter.

```
SELECT COUNT(*) AS total_employees
FROM employees
WHERE LEFT(name, 1) = RIGHT(name, 1);
```



04

How to find the department that has the highest average salary?

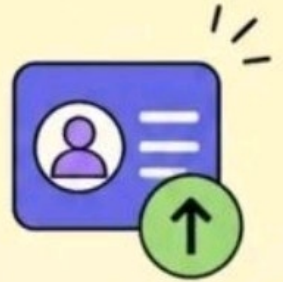
```
SELECT department_id, AVG(salary) AS avg_salary
FROM employees
GROUP BY department_id
ORDER BY avg_salary DESC
LIMIT 1;
```



05

How to find employees who earn more than the average salary of their department?

```
SELECT e.employee_id, e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (SELECT AVG(salary)
                  FROM employees
                  WHERE department_id = e.department_id);
```



06

How to find the top 3 highest paid employees?

```
SELECT employee_id, name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;
```



07

How to find duplicate emails in the employees table?

```
SELECT email, COUNT(*) AS total_count
FROM employees
GROUP BY email
HAVING COUNT(*) > 1;
```



08

How to find the cumulative (running total) salary of employees ordered by salary?

```
SELECT employee_id, name, salary,
       SUM(salary) OVER (ORDER BY salary) AS running_total
FROM employees
ORDER BY salary;
```



09

Find the employees who earn more than the average salary of their department.

```
SELECT e.employee_id, e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
    SELECT AVG(salary)
    FROM employees
    WHERE department_id = e.department_id
);
```

Output

| employee_id | name   | salary | department_id |
|-------------|--------|--------|---------------|
| 101         | John   | 90000  | 1             |
| 105         | Alice  | 75000  | 2             |
| 109         | Robert | 85000  | 1             |

10

Find duplicate records in a table based on a specific column.

```
SELECT email, COUNT(*) AS count
FROM employees
GROUP BY email
HAVING COUNT(*) > 1;
```

Output

| email             | count |
|-------------------|-------|
| john@example.com  | 2     |
| alice@example.com | 2     |

11

Find the top 3 highest paid employees in each department.

```
SELECT employee_id, name, salary, department_id
FROM (
    SELECT e.*,
        ROW_NUMBER() OVER (PARTITION BY department_id ORDER BY salary DESC) AS rn
    FROM employees e
) t
WHERE rn <= 3
ORDER BY department_id, salary DESC;
```

Output

| employee_id | name    | salary | department_id |
|-------------|---------|--------|---------------|
| 101         | John    | 90000  | 1             |
| 109         | Robert  | 85000  | 1             |
| 110         | Michael | 70000  | 1             |
| ...         | ...     | ...    | ...           |
| 105         | Alice   | 75000  | 2             |
| 107         | Sophia  | 65000  | 2             |
| 111         | David   | 60000  | 2             |

12

Find departments that have more than 5 employees.

```
SELECT department_id, COUNT(*) AS total_employees
FROM employees
GROUP BY department_id
HAVING COUNT(*) > 5;
```

Output

| department_id | total_employees |
|---------------|-----------------|
| 1             | 8               |
| 2             | 6               |
| 3             | 7               |

**13** List employees who do not belong to any department.

```
SELECT e.employee_id, e.name
FROM employees e
LEFT JOIN departments d
ON e.department_id = d.department_id
WHERE d.department_id IS NULL;
```

Output

| employee_id | name    |
|-------------|---------|
| 112         | William |
| 113         | Olivia  |

**14** Find the total salary paid by each department.

```
SELECT d.department_id, d.department_name,
       SUM(e.salary) AS total_salary
FROM departments d
LEFT JOIN employees e
ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;
```

Output

| department_id | department_name | total_salary |
|---------------|-----------------|--------------|
| 1             | Development     | 240000       |
| 2             | HR              | 140000       |
| 3             | Sales           | 215000       |
| 4             | Support         | 125000       |

**15** Find employees who earn more than the average salary of the company.

```
SELECT employee_id, name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Output

| employee_id | name    | salary |
|-------------|---------|--------|
| 101         | John    | 90000  |
| 103         | Robert  | 85000  |
| 105         | Alice   | 75000  |
| 106         | Michael | 70000  |

**16** Find the month-wise count of employees who joined.

```
SELECT DATE_FORMAT(join_date, '%Y-%m') AS join_month,
       COUNT(*) AS total_employees
FROM employees
GROUP BY join_month
ORDER BY join_month;
```

Output

| join_month | total_employees |
|------------|-----------------|
| 2018-06    | 2               |
| 2019-04    | 3               |
| 2020-01    | 2               |
| 2020-03    | 1               |

17

Find customers who have placed orders but never returned any product.

```
SELECT c.customer_id, c.customer_name
FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o
              WHERE o.customer_id = c.customer_id)
AND NOT EXISTS (SELECT 1 FROM returns r
               WHERE r.customer_id = c.customer_id);
```

Output

| customer_id | customer_name   |
|-------------|-----------------|
| 101         | John Doe        |
| 103         | Michael Johnson |
| 107         | Chris Martin    |
| 110         | Olivia Brown    |

18

Find the month-over-month sales growth percentage.

```
SELECT sale_month,
       total_sales,
       LAG(total_sales) OVER (ORDER BY sale_month) AS prev_month_sales,
       ROUND(((total_sales - LAG(total_sales) OVER (ORDER BY sale_month))
              / LAG(total_sales) OVER (ORDER BY sale_month)) * 100, 2) AS growth_pct
FROM monthly_sales;
```

Output

| sale_month | total_sales | prev_month_sales | growth_pct |
|------------|-------------|------------------|------------|
| 2024-01    | 120000      | NULL             | NULL       |
| 2024-02    | 150000      | 120000           | 25.00      |
| 2024-03    | 180000      | 150000           | 20.00      |
| 2024-04    | 210000      | 180000           | 16.67      |

19

Find the top 3 products by total revenue in each category.

```
SELECT * FROM (
  SELECT category_id, product_id, product_name, total_revenue,
         ROW_NUMBER() OVER (PARTITION BY category_id
                             ORDER BY total_revenue DESC) AS rn
  FROM product_revenue
) t
WHERE rn <= 3
ORDER BY category_id, total_revenue DESC;
```

Output

| category_id | product_id | product_name | total_revenue |
|-------------|------------|--------------|---------------|
| 1           | 101        | Laptop       | 250000        |
| 1           | 105        | Monitor      | 180000        |
| 1           | 110        | Keyboard     | 120000        |
| ...         | ...        | ...          | ...           |
| 3           | 205        | Shirt        | 95000         |
| 3           | 203        | Jeans        | 85000         |
| 3           | 207        | Shoes        | 70000         |

20

Find employees whose salary is higher than the average salary in their department.

```
SELECT e.employee_id, e.name, e.salary, e.department_id
FROM employees e
WHERE e.salary > (
  SELECT AVG(salary) FROM employees
  WHERE department_id = e.department_id
);
```

Output

| employee_id | name   | salary | department_id |
|-------------|--------|--------|---------------|
| 102         | Alice  | 85000  | 1             |
| 104         | David  | 90000  | 1             |
| 109         | Sophia | 75000  | 2             |
| 112         | James  | 88000  | 3             |

21

Find the customers who spent more than 1000 in total.

```
SELECT c.customer_id, c.customer_name,
       SUM(o.amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING SUM(o.amount) > 1000;
```

Output



| customer_id | customer_name   | total_spent |
|-------------|-----------------|-------------|
| 101         | John Doe        | 1200        |
| 103         | Michael Johnson | 2300        |
| 107         | Chris Martin    | 1550        |
| 110         | Olivia Brown    | 2250        |

22

List orders that have not been shipped yet.

```
SELECT order_id, customer_id, order_date, status
FROM orders
WHERE status = 'Pending';
```

Output



| order_id | customer_id | order_date | status  |
|----------|-------------|------------|---------|
| 1002     | 101         | 2024-05-12 | Pending |
| 1005     | 103         | 2024-05-18 | Pending |
| 1008     | 107         | 2024-05-21 | Pending |
| 1011     | 110         | 2024-05-25 | Pending |

23

Find the number of orders placed by each customer.

```
SELECT c.customer_id, c.customer_name,
       COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_orders DESC;
```

Output



| customer_id | customer_name   | total_orders |
|-------------|-----------------|--------------|
| 103         | Michael Johnson | 7            |
| 101         | John Doe        | 5            |
| 110         | Olivia Brown    | 4            |
| 107         | Chris Martin    | 3            |
| 104         | Emily Davis     | 2            |

24

Find the latest order date for each customer.

```
SELECT c.customer_id, c.customer_name,
       MAX(o.order_date) AS latest_order_date
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY latest_order_date DESC;
```

Output



| customer_id | customer_name   | latest_order_date |
|-------------|-----------------|-------------------|
| 103         | Michael Johnson | 2024-05-24        |
| 110         | Olivia Brown    | 2024-05-25        |
| 101         | John Doe        | 2024-05-22        |
| 107         | Chris Martin    | 2024-05-21        |
| 104         | Emily Davis     | 2024-05-19        |

25

Find the highest salary in each department.

```
SELECT d.department_id, d.department_name,
       MAX(e.salary) AS highest_salary
FROM departments d
JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name
ORDER BY highest_salary DESC;
```

Output



| department_id | department_name | highest_salary |
|---------------|-----------------|----------------|
| 1             | Development     | 95000          |
| 3             | Sales           | 90000          |
| 2             | HR              | 65000          |
| 4             | Support         | 45000          |

26

Find duplicate emails from the employees table.

```
SELECT email, COUNT(*) AS occurrence
FROM employees
GROUP BY email
HAVING COUNT(*) > 1;
```

Output



| email                | occurrence |
|----------------------|------------|
| john.doe@gmail.com   | 2          |
| alice.work@gmail.com | 3          |
| mike123@gmail.com    | 2          |
| emma.dev@gmail.com   | 2          |

27

List the second highest salary from the employees table.

```
SELECT MAX(salary) AS second_highest_salary
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Output



| second_highest_salary |
|-----------------------|
| 90000                 |

28

Show the employees who manage more than 5 employees.

```
SELECT m.employee_id, m.name,
       COUNT(e.employee_id) AS team_size
FROM employees m
LEFT JOIN employees e ON m.employee_id = e.manager_id
GROUP BY m.employee_id, m.name
HAVING COUNT(e.employee_id) > 5;
```

Output



| employee_id | name            | team_size |
|-------------|-----------------|-----------|
| 101         | John Doe        | 8         |
| 103         | Michael Johnson | 6         |
| 107         | Chris Martin    | 7         |
| 110         | Olivia Brown    | 9         |

29

Find products that have never been ordered.

```
SELECT p.product_id, p.product_name
FROM products p
LEFT JOIN orders o
ON p.product_id = o.product_id
WHERE o.order_id IS NULL;
```

Output



| product_id | product_name |
|------------|--------------|
| 104        | Tablet       |
| 105        | Smartwatch   |
| 106        | Headphones   |

30

Find the total sales amount for each product.

```
SELECT p.product_name,
       SUM(o.amount) AS total_sales
FROM products p
JOIN orders o ON p.product_id = o.product_id
GROUP BY p.product_name
ORDER BY total_sales DESC;
```

Output



| product_name | total_sales |
|--------------|-------------|
| Laptop       | 135000      |
| Smartphone   | 95000       |
| Keyboard     | 45000       |
| Mouse        | 15000       |

31

Find customers who have placed more than 3 orders.

```
SELECT c.customer_id, c.customer_name,
       COUNT(o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
HAVING COUNT(o.order_id) > 3;
```

Output



| customer_id | customer_name   | total_orders |
|-------------|-----------------|--------------|
| 101         | John Doe        | 5            |
| 103         | Michael Johnson | 6            |
| 107         | Chris Martin    | 4            |
| 110         | Olivia Brown    | 7            |

32

Find the average order amount for each customer.

```
SELECT c.customer_id, c.customer_name,
       ROUND(AVG(o.amount), 2) AS avg_order_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY avg_order_amount DESC;
```

Output



| customer_id | customer_name   | avg_order_amount |
|-------------|-----------------|------------------|
| 103         | Michael Johnson | 31500.75         |
| 110         | Olivia Brown    | 28500.50         |
| 101         | John Doe        | 22600.33         |
| 107         | Chris Martin    | 18750.25         |

33

Find the category that has the highest average product price.

```
SELECT category_id, AVG(price) AS avg_price
FROM products
GROUP BY category_id
ORDER BY avg_price DESC
LIMIT 1;
```

Output



| category_id | avg_price |
|-------------|-----------|
| 2           | 12500.00  |

34

List customers who have not placed any orders.

```
SELECT c.customer_id, c.customer_name
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
WHERE o.order_id IS NULL;
```

Output



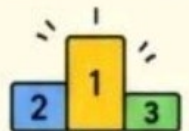
| customer_id | customer_name |
|-------------|---------------|
| 104         | Emily Davis   |
| 111         | Daniel Wilson |
| 115         | Sophia Taylor |

35

Find the top 2 customers by total amount spent.

```
SELECT c.customer_id, c.customer_name,
       SUM(o.amount) AS total_spent
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.customer_name
ORDER BY total_spent DESC
LIMIT 2;
```

Output



| customer_id | customer_name   | total_spent |
|-------------|-----------------|-------------|
| 101         | John Doe        | 48500       |
| 103         | Michael Johnson | 36200       |

36

Find the orders where the amount is greater than the average order amount.

```
SELECT order_id, customer_id, order_date, amount
FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders);
```

Output



| order_id | customer_id | order_date | amount |
|----------|-------------|------------|--------|
| 1003     | 101         | 2024-05-10 | 2500   |
| 1005     | 103         | 2024-05-18 | 3200   |
| 1007     | 107         | 2024-05-21 | 4100   |
| 1010     | 110         | 2024-05-25 | 2800   |

37

Find the months in which we had more than 10 orders.

```
SELECT DATE_FORMAT(order_date, '%Y-%m') AS month,
       COUNT(*) AS total_orders
FROM orders
GROUP BY month
HAVING COUNT(*) > 10
ORDER BY month;
```

Output



| month   | total_orders |
|---------|--------------|
| 2024-04 | 12           |
| 2024-05 | 18           |
| 2024-06 | 15           |
| 2024-07 | 11           |

38

Find the employees who earn more than the average salary.

```
SELECT employee_id, name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Output



| employee_id | name            | salary |
|-------------|-----------------|--------|
| 101         | John Doe        | 75000  |
| 103         | Michael Johnson | 65000  |
| 105         | Olivia Brown    | 70000  |
| 107         | Chris Martin    | 62000  |

39

Find the categories that have no products.

```
SELECT c.category_id, c.category_name
FROM categories c
LEFT JOIN products p
ON c.category_id = p.category_id
WHERE p.product_id IS NULL;
```

Output



| category_id | category_name |
|-------------|---------------|
| 5           | Toys          |
| 6           | Books         |
| 7           | Beauty        |

40

Show the running total of sales for each order date.

```
SELECT order_date,
       SUM(amount) AS daily_total,
       SUM(SUM(amount)) OVER (ORDER BY order_date)
       AS running_total
FROM orders
GROUP BY order_date
ORDER BY order_date;
```

Output



| order_date | daily_total | running_total |
|------------|-------------|---------------|
| 2024-05-10 | 2500        | 2500          |
| 2024-05-11 | 3200        | 5700          |
| 2024-05-12 | 4100        | 9800          |
| 2024-05-13 | 2800        | 12600         |